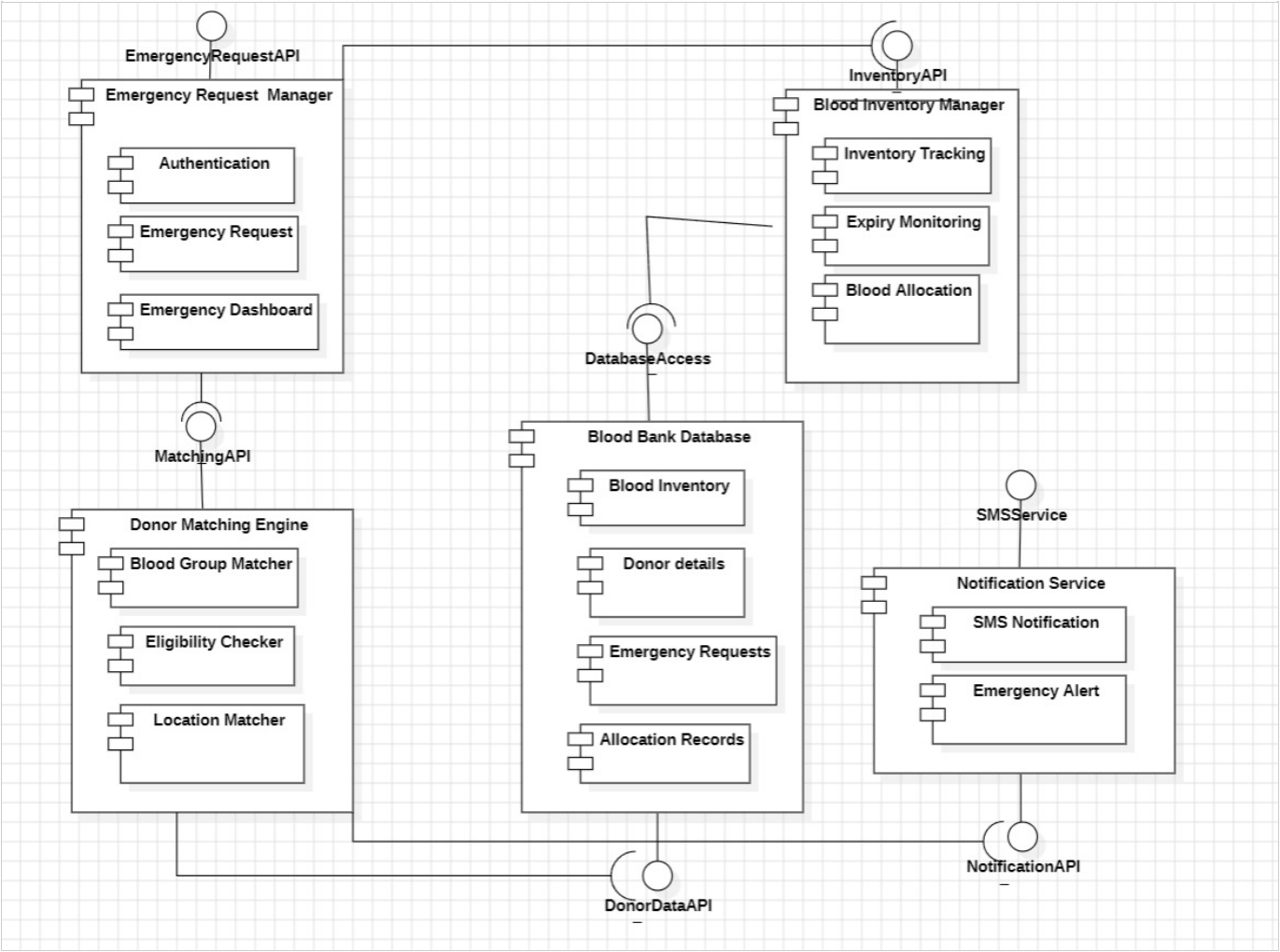
LAB 3 – COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Problem Statement #12 — Healthcare & Telemedicine: Blood Bank Inventory & Emergency Donor Matcher

Student: Satvik Das **SRN:** PES2UG24CS448 **Architecture:** Microservices Architecture



LAB 3 – ARCHITECTURE SELECTION & JUSTIFICATION

Student: Satvik Das | **SRN:** PES2UG24CS448

We chose Microservices Architecture for the Blood Bank Inventory & Emergency Donor Matcher System.

Architectural Choice

The system is broken down into independent services: Emergency Request Manager, Blood Inventory Manager, Donor Matching Engine, Notification Service, and Blood Bank Database, each exposing its own API instead of sharing internal logic directly. We looked at a Layered architecture as well, but a single business layer handling emergency requests, donor matching, and notifications together didn't seem like a good fit once those three things need to scale and fail on their own.

Reason 1 – Independent scaling during emergencies

Emergency blood requests don't come in at a constant rate. During a shortage or a mass casualty situation, the Emergency Request Manager and Notification Service suddenly have to handle a lot more traffic, while inventory tracking and expiry monitoring stay roughly the same. Because these are separate services, we can scale up just the request handling and notification services when this happens instead of scaling the whole application, which is more practical and doesn't waste resources during normal periods.

Reason 2 – Fault isolation between matching and inventory

The Donor Matching Engine and the Blood Inventory Manager do very different jobs. If something goes wrong with donor matching, for example a location query failing, the Blood Inventory Manager should still be able to track expiry dates and update stock without being affected. Since inventory accuracy is critical for a blood bank, keeping these as separate services means one service failing doesn't bring down the other.

Security Advantage

Each service only exposes what it needs to through a defined interface, not its internal data or logic. Donor details are kept inside the Donor Matching Engine and can only be reached through the DonorDataAPI, so the Emergency Request Manager, which is the part of the system exposed to outside requests, never has direct access to donor records. This means that even if the request-handling service were compromised, the attacker still wouldn't have a direct path to donor information or the inventory database.

Performance Benefit

Since matching, inventory, and notification are handled by separate services, sending SMS alerts to compatible donors doesn't have to wait for inventory checks or other operations to finish first. The Notification Service can dispatch alerts while the Blood Inventory Manager and Donor Matching Engine keep working on their own tasks. This kind of concurrency is what makes it realistic to notify compatible donors quickly instead of processing every step of the request one after another in a single application.